Machine Learning in Production

Lecture 1: From Models to AI-Enabled Systems

Son Nguyen
VNU University of Engineering and Technology

Today’s release decision

Meet UniGuide

- Your team has built **UniGuide**, an AI assistant for UET
- A student can ask:
  - “Can I graduate next semester?”
  - “Which course satisfies this prerequisite?”
  - “What happens if I withdraw from this course?”
  - “When is the scholarship application due?”





Evaluation report looks strong

- Most benchmark questions receive acceptable answers.
- The assistant usually cites a relevant document.
- Median response time is acceptable in the test environment.
- Students in a pilot say the tool is convenient.

Do you approve the release?

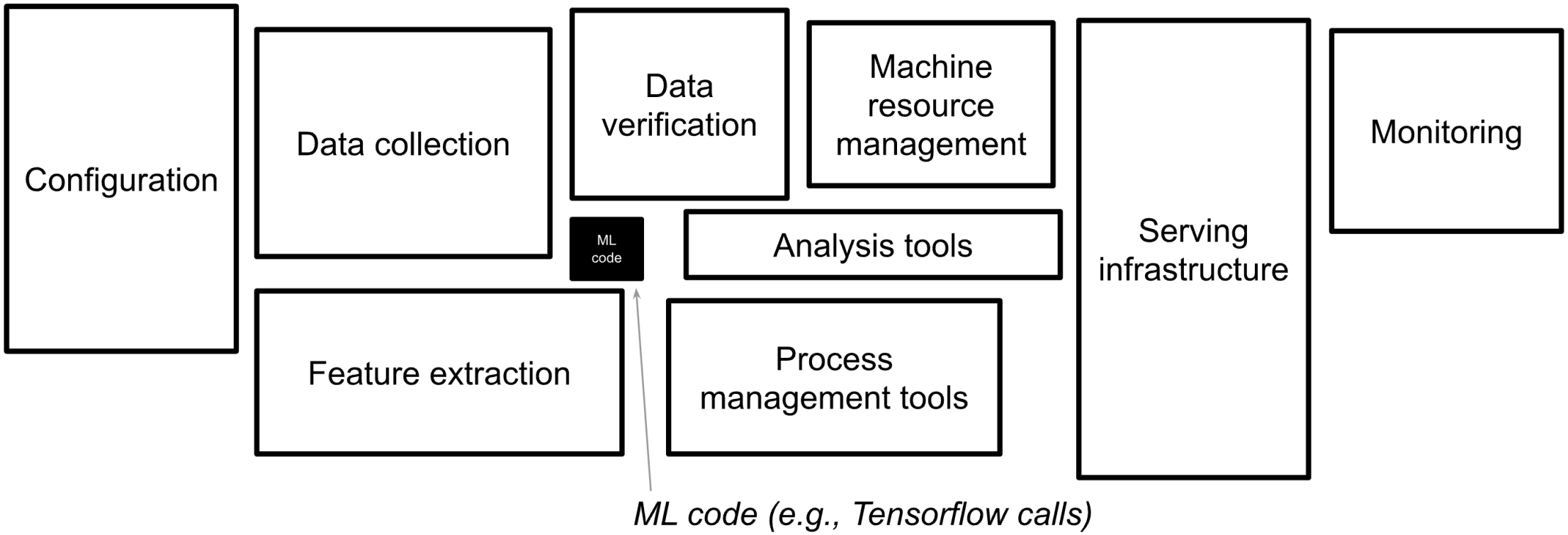


New facts are arriving…

1. The benchmark was assembled from last year’s frequently asked questions. A revised academic regulation took effect last week.
2. The retrieval system sometimes finds a departmental guidance page rather than the official university regulation. Both pages sound authoritative.
3. The assistant can draft a withdrawal request. A new feature allows it to submit the request after the student confirms.
4. Some students ask in Vietnamese and English in the same message. These requests were uncommon in the evaluation set.
5. During registration week, traffic may be many times higher than the pilot traffic. The external LLM charges per token and can change service limits.
6. The interface presents one fluent answer. It does not show uncertainty, conflicting policies, or an obvious route to a human adviser.

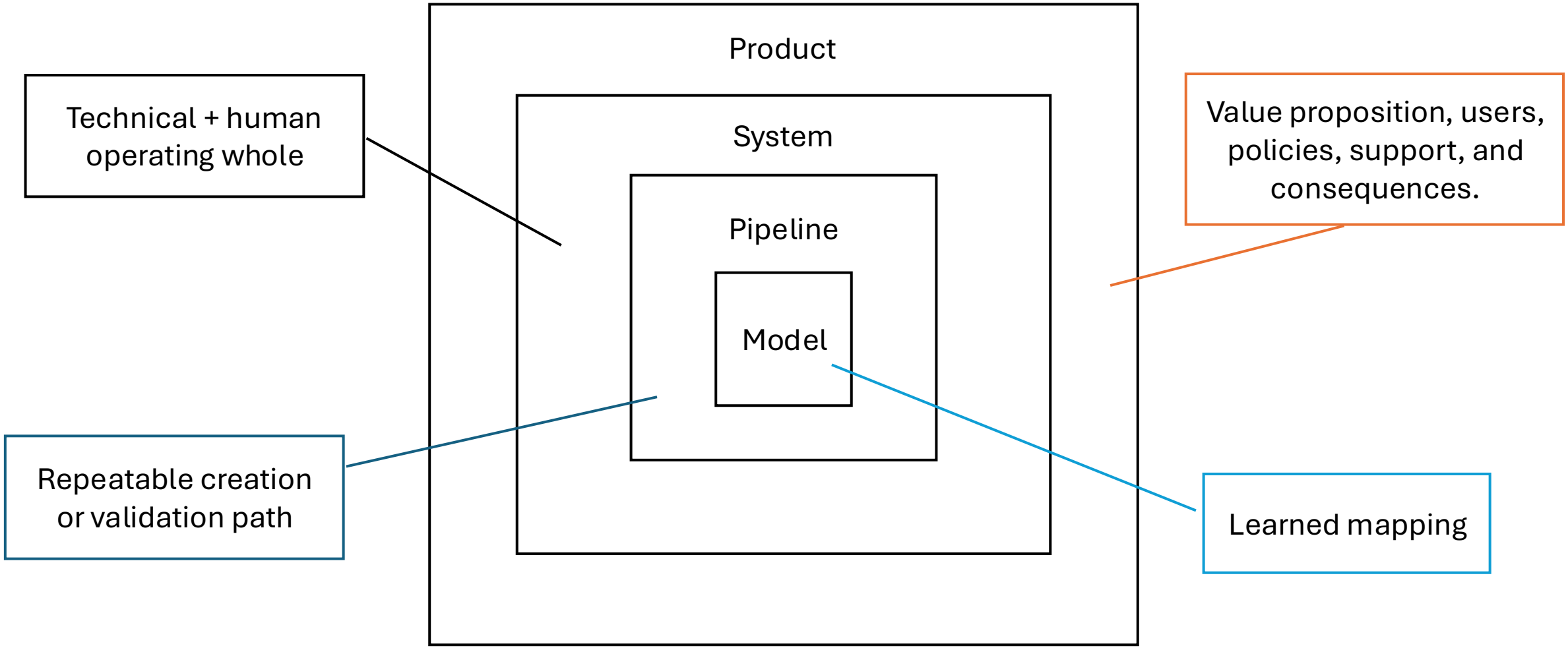
Discussion

ML Model is just a small part…



What exactly are we engineering?

Four boundaries



AI/ML Model

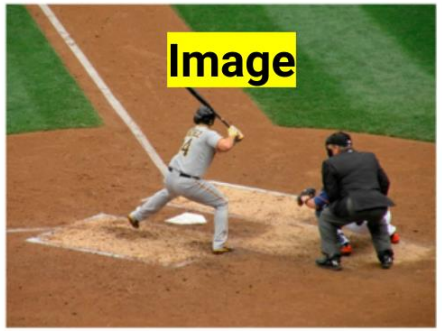
- For UniGuide, the “model” could refer to a language model that generates an answer.
  - That is already an oversimplification
  - The behavior may depend on retrieval, prompts, tool descriptions, decoding settings, provider-side updates, and previous conversation context.
- A model-centered question asks:
  - How accurate is it?
  - How well does it generalize?
  - How does it compare with a baseline?
  - Where does it fail?
  - How fast and costly is inference?







Traditional Software



Algorithm

Caption

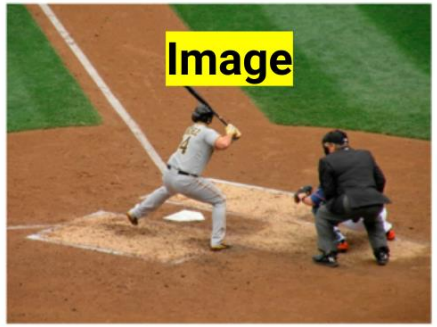
The man at bat readies to swing at the pitch while the umpire looks on.

AI/ML Model



Training Data (Examples)

ML algorithm



Image

Model

Caption

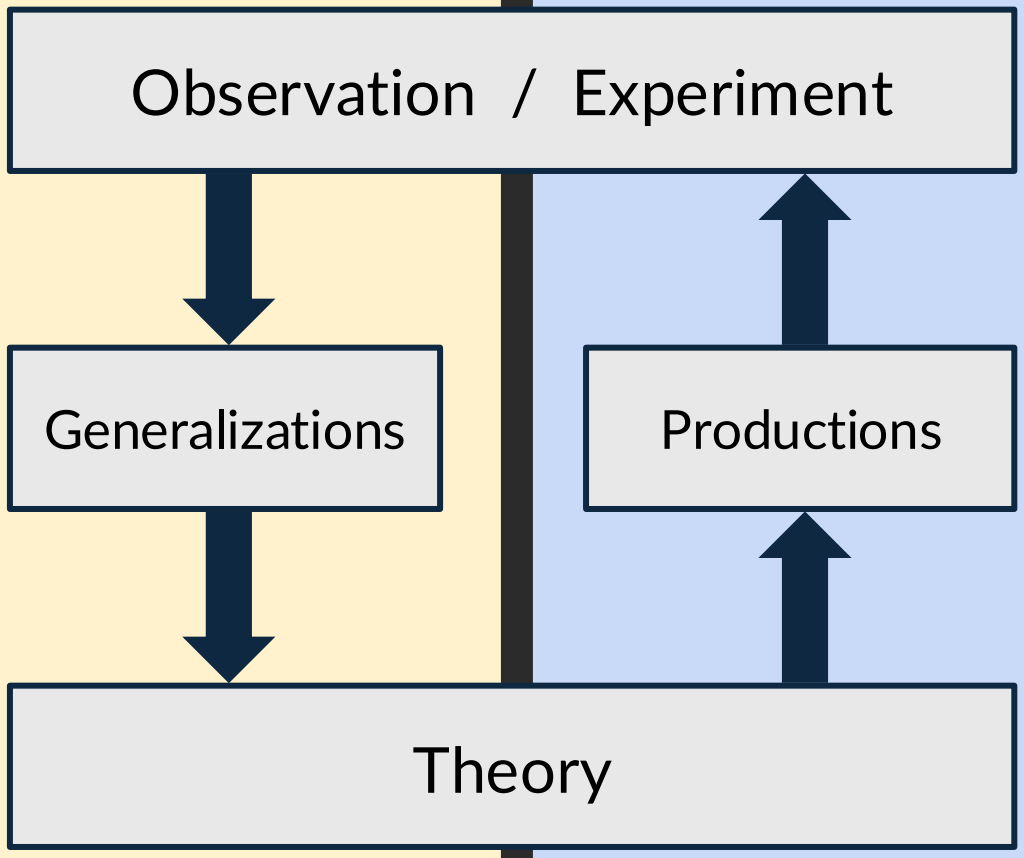
The man at bat readies to swing at the pitch while the umpire looks on.

Inductive Reasoning

Deriving theories from observations

Strong evidence suggests a rule

Science-y reasoning



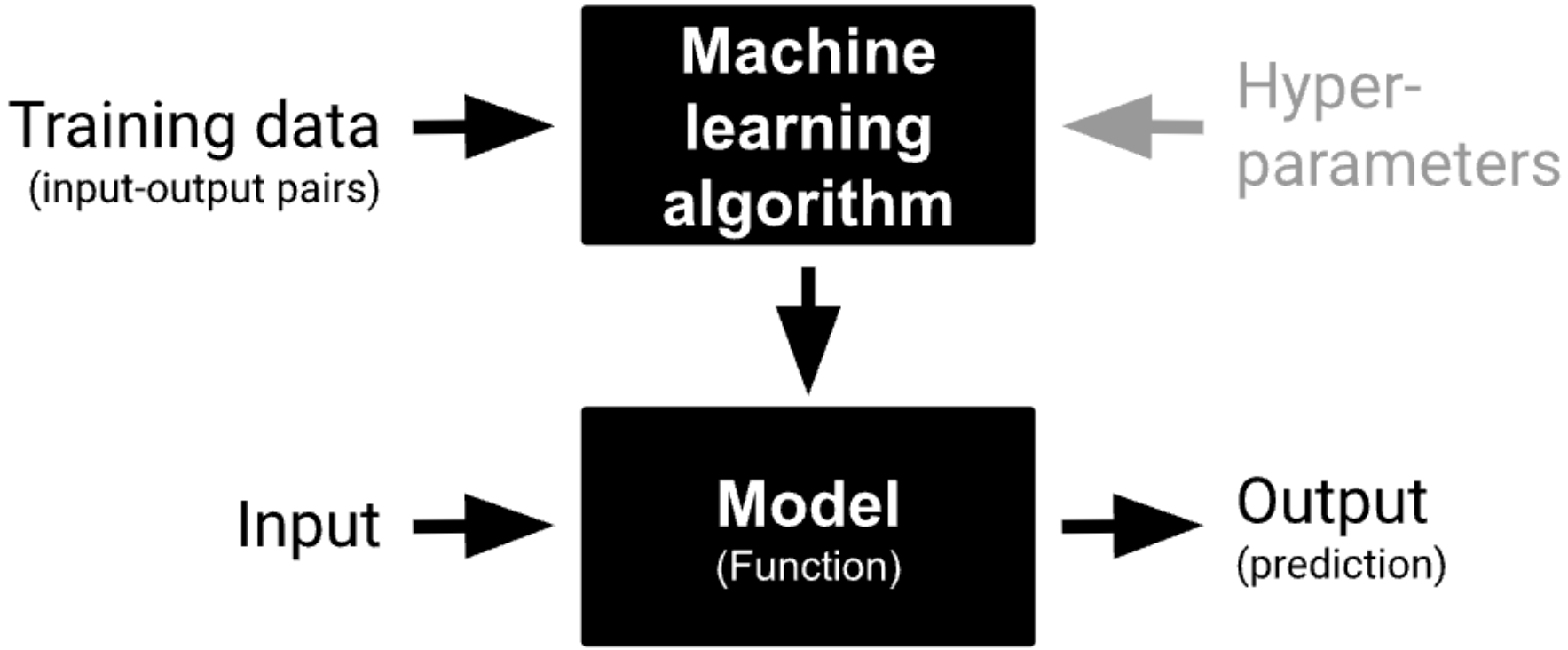
Deductive Reasoning

Combining logical statements

Proving theorems from axioms

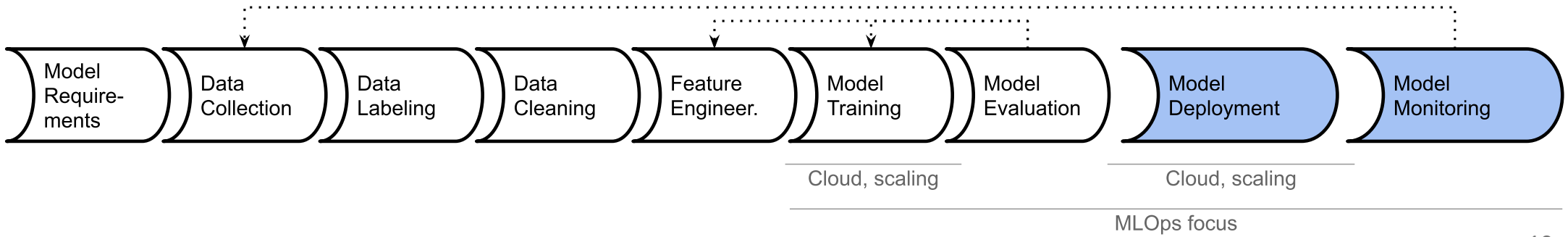
Math-y reasoning

Conceptual steps of ML



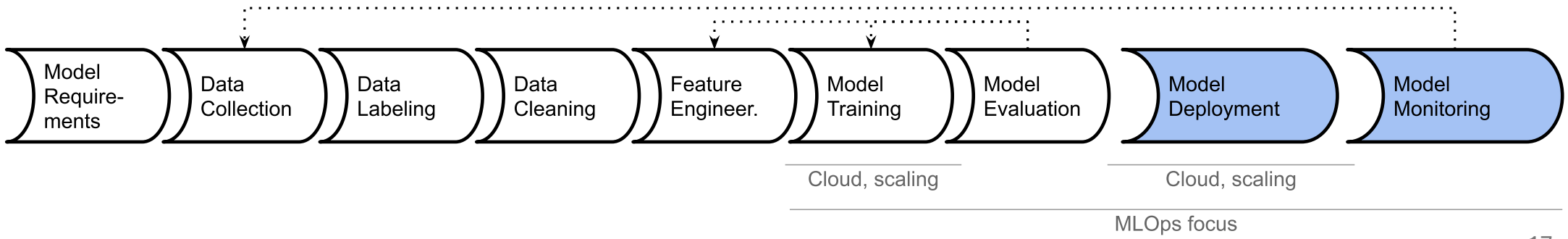
AI/ML Pipeline

- A traditional ML pipeline may include:
  - collecting and labeling data;
  - validating and transforming data;
  - feature engineering;
  - training;
  - evaluation;
  - packaging and deployment;
  - monitoring and retraining.
- For an LLM application, the pipeline may also manage:
  - prompts and system instructions;
  - retrieval documents and indexes;
  - evaluation and test conversations;
  - model or provider selection;
  - tool definitions and permissions;
  - safety policies;
  - experiment configuration.



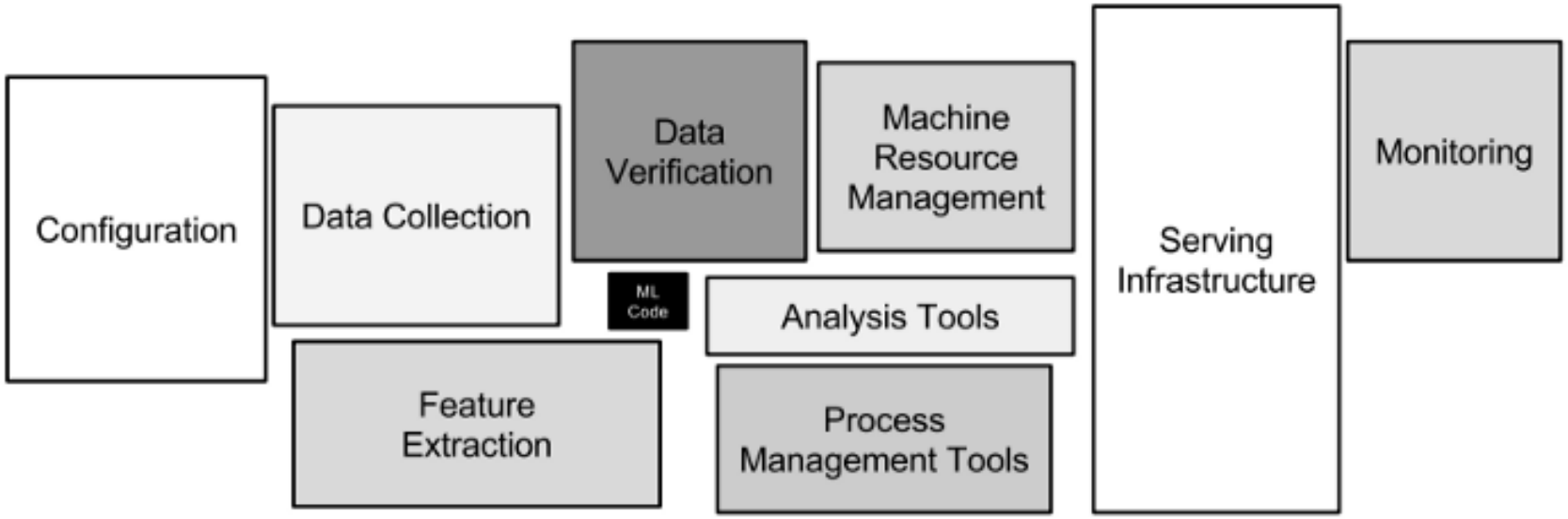
The pipeline answers…

- Can we reproduce this model or configuration?
- Can we test changes before release?
- Can we promote or reject a candidate automatically?
- Can we trace the data and configuration used?
- Can we roll back?



ML-powered System

- The system includes the model and pipeline, plus other components…



For UniGuide, what components the system may contain?

A system-centered question asks

- What happens after the model produces an output?
- Can another component check, constrain, defer, or reverse it?
- Which failures can combine?
- Which human has information, time, authority, and responsibility to intervene?
- What does the system observe from the world, and how does it change that world?



Product

Could the system work as specified and the product still fail?

Data scientists' challenges

- **Infrastructure:** Understand chat bot infrastructure and its capabilities
- **Knowing topics:** Identify what users talk about, train/test concepts with past chat logs
- **Guiding conversations:** Supporting open-ended conversations requires detecting what's on topic and finding a good response; intent-topic modeling
  - *Is talk about parents and children on topic when discussing divorce?*
  - Data gathering/labeling very challenging -- too many corner cases

Stepping Back: What are the product' goals?



DS lead

Maybe we need to think about it like an 80/20 rule. In some cases, it works well, but for some, it is harder. 80% everything is fine, and in the remaining 20%, we try to do our best

The trouble is how to automatically recognize what is 80 and what is 20

It is harder than it sounds. One of the models is a matching model trained on pairs of legal questions and answers. 60K of them. It seems large but is small for ML

That's a lot. Can it answer a question about say visa renewal?

If there exists a question like that in training data, then yes. But with just 60K, the model can easily overfit, and then for anything outside, it would just fail

I see what you are saying. Edge cases are interesting from an academic perspective, but for a business the first and foremost thing is value. You are trying to solve an interesting problem. I get it. But I feel that you may have already solved it enough to gain business value



Data Scientist



Customer

Production' goals for Chatbot

- Collect user data to sell to lawyers
- Signal technical competency to lawyers
- Acceptable to fail: Too complicated for self-help, connect with lawyer
- Solving edge cases not important



Edge cases are important, but the end goal is user information, monetizing user data. We are building a legal self-help chatbot, but a major business use case is to tell people: 'here, talk to this lawyer.' We do want to connect them with a lawyer. Even for 20%, when our bot fails, we tell users that the problem cannot be done through self-help. Let us get you a lawyer, right? That is what we wanted in the first place.

Why familiar SE is necessary but insufficient

The specification problem

Traditional Software

```
/**
 * Return the sum of all values
 * @ensures \result = \sum int i; 0 <= i < …
 */
int sum(int[] values);
```

AI-Enabled Software

```
/**
 * Detects large anomalous dark regions in grayscale X-ray baggage
 * scans that may indicate lead shielding, hidden compartments, or
 * dense materials.
 * <p>Algorithm: Gaussian blur → binary threshold → connected
 * component analysis (8-connectivity) → filter by minimum area →
 * compute region metrics
 *
 * @param 2D grayscale pixel int. [height][width], range [0, 255]
 * [...]
 */
public List<DarkRegion> detectLargeDarkSpots(int[][] imageData,
    int darknessThreshold, int minAreaPixels, double pixelsPerMm)
```

ML Models Make Mistakes

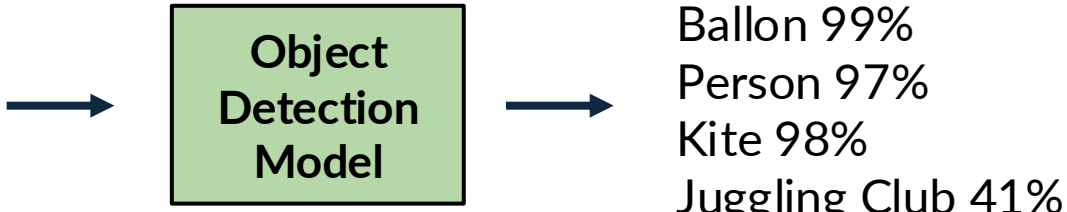
...often in unexpected ways

- Hard to foresee and capture because no spec
- What does it mean to be correct? Can only evaluate whether it works well enough (on average) on some test data!



ML Model = Unreliable Function





- No guarantees, may make mistakes, confidence unreliable
- Model often inscrutable, opaque
- Evaluated in terms of accuracy, not correctness

Accuracy

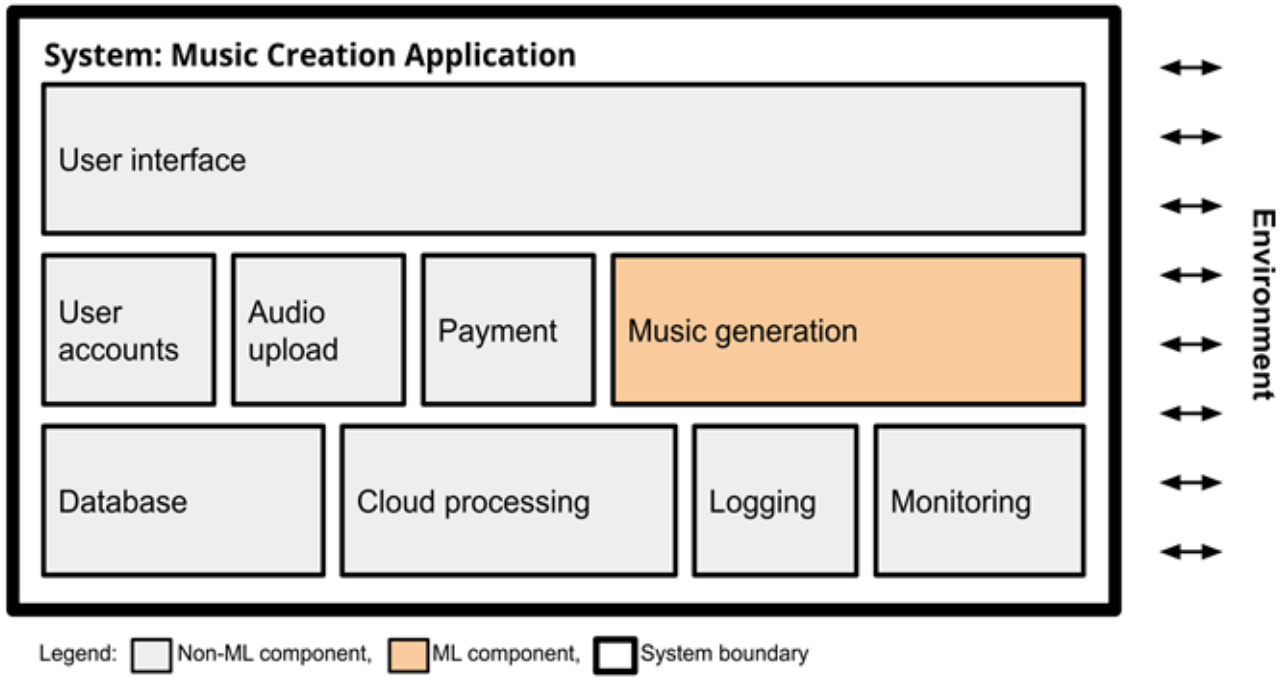
≠

Correctness

≠

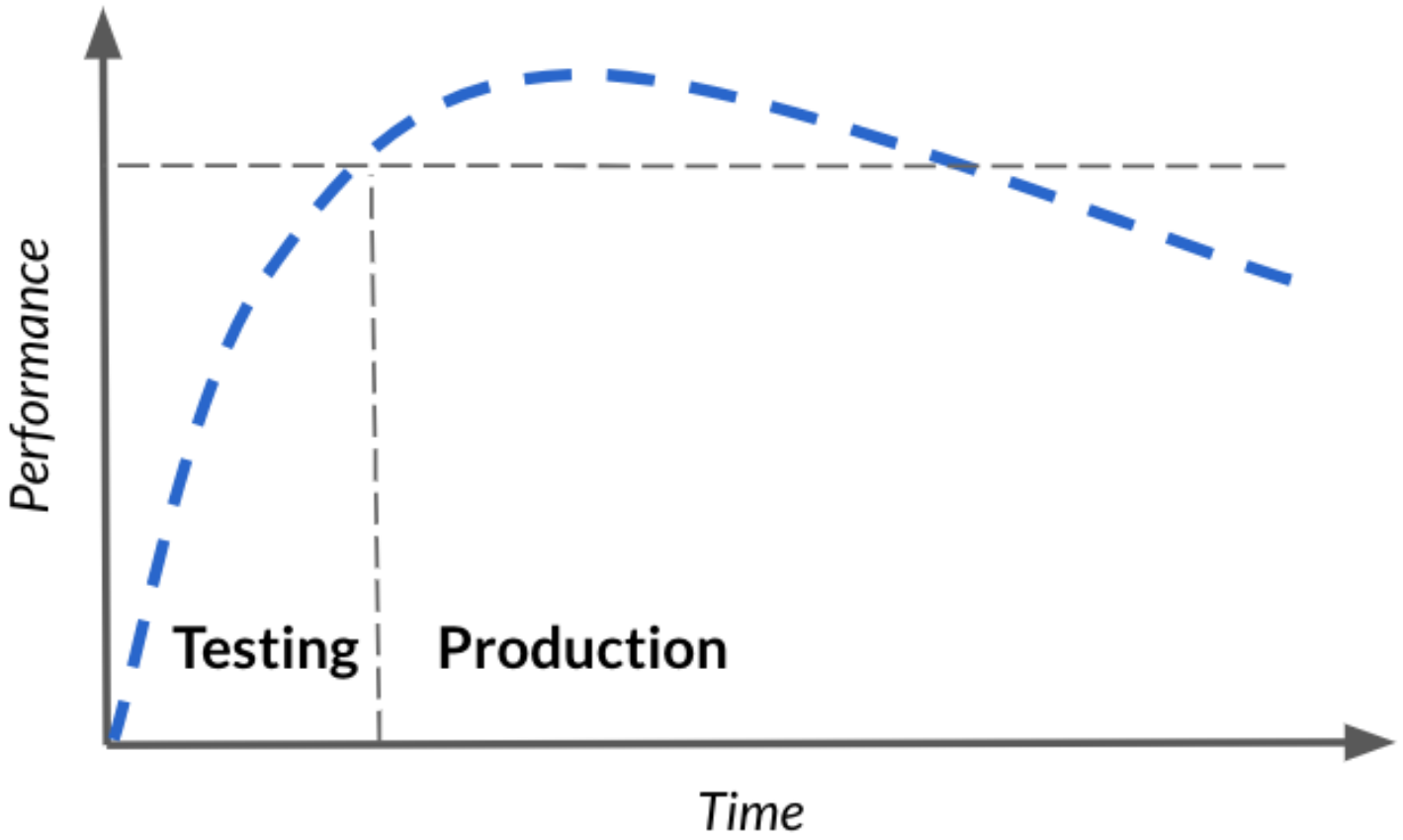
Usefulness

Interaction with the environment

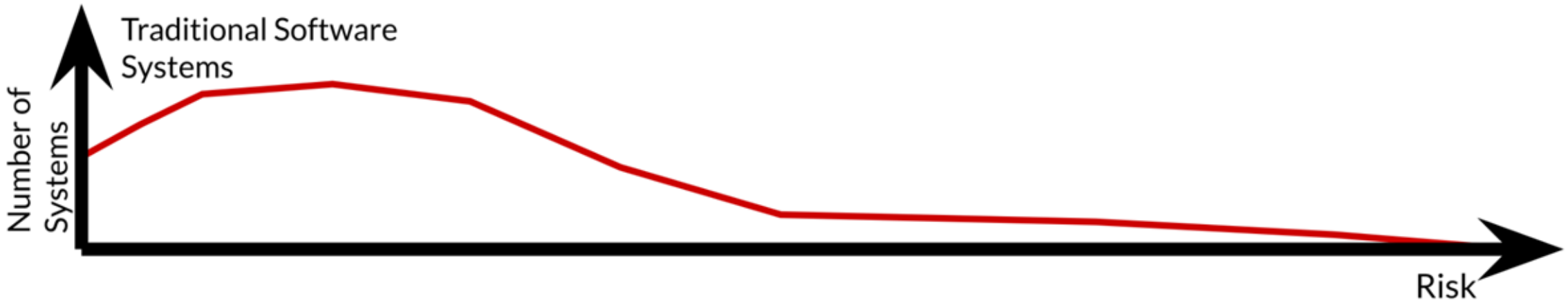


Our system must be able to tolerate some incorrect predictions, and be aware of how it might influence the world…

“Change”, yeah, again!



Complexity

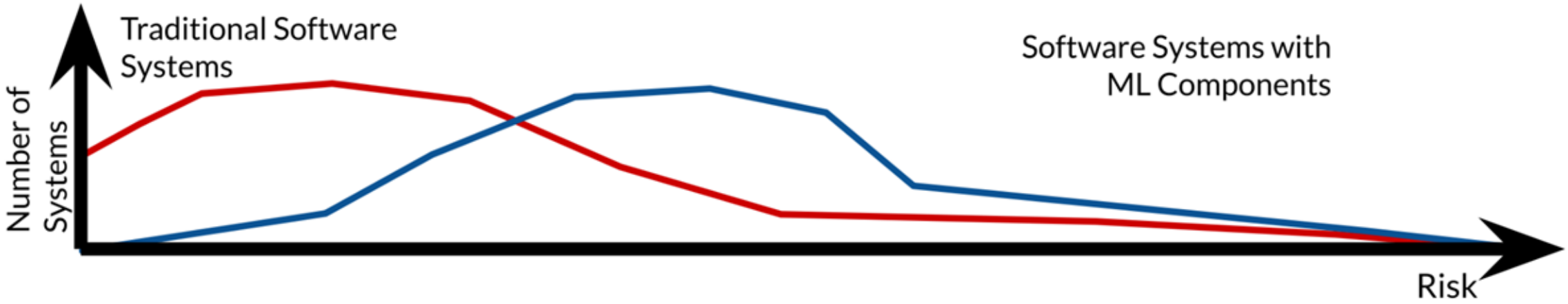


Restaurant website
Video streaming service
Podcast hosting
Conference website

Electric scooters
Medical records
Payment software

Nuclear power plant
Space exploration

Complexity



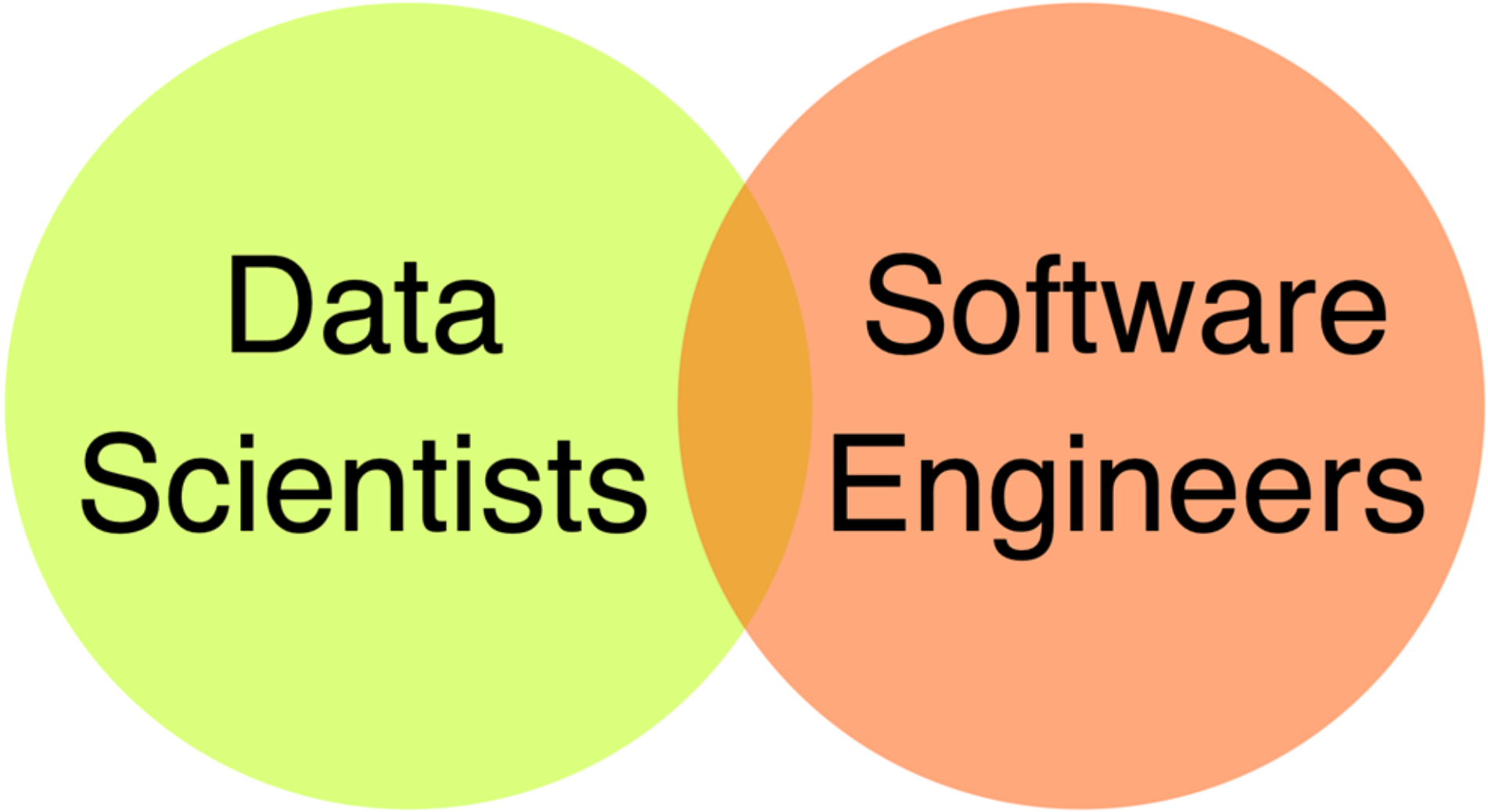
Game AI
Traffic planning
Photo filters

Photo search
Social media feeds
Sidewalk robot delivery
Smart home devices
Automated transcription

Autonomous vehicles
Predictive policing
Deep Fakes

The coordination problem





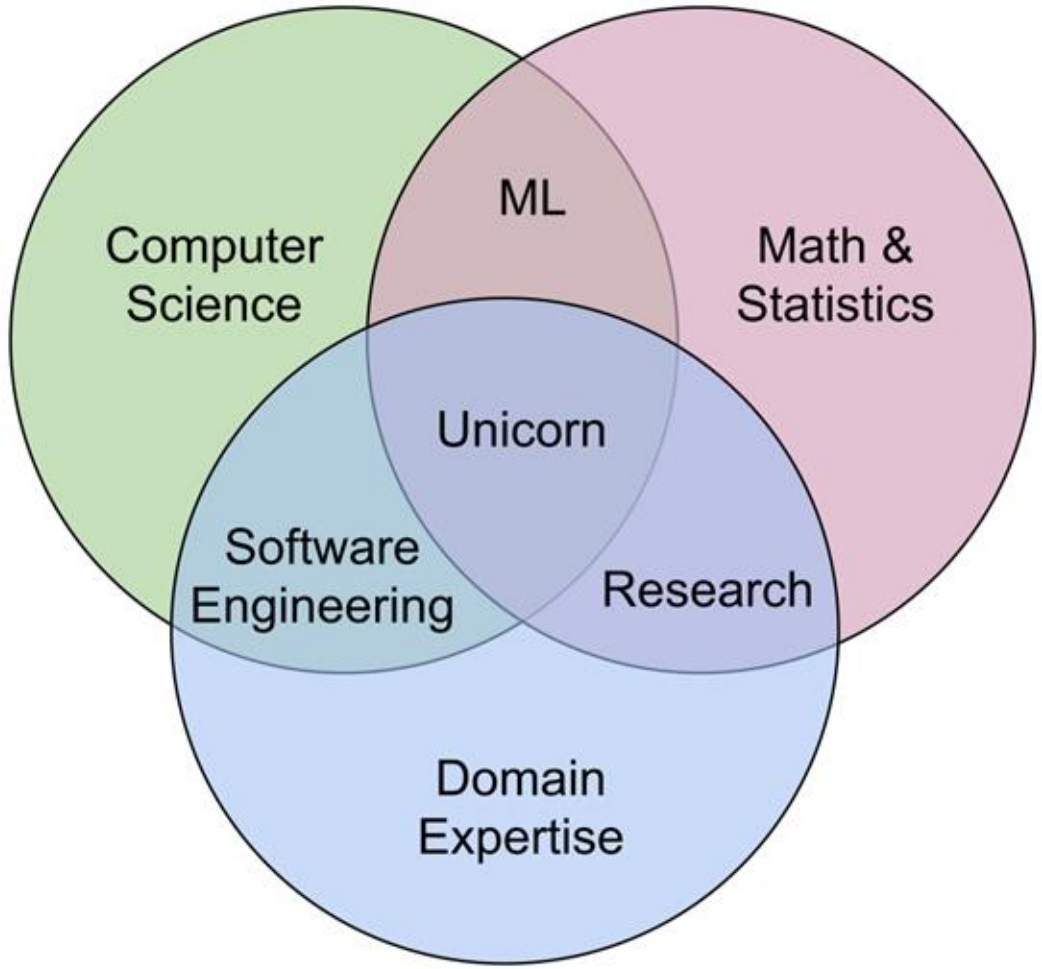
and Data engineers + Domain specialists + Operators + Business team + Project managers + Designers, UI Experts + Safety, security specialists + Lawyers + Social Scientists + ...

Data scientist

- Often fixed dataset for training and evaluation (e.g., PBS interviews)
- Focused on accuracy
- Prototyping, often Jupyter notebooks or similar
- Expert in modeling techniques and feature engineering
- Model size, updateability, implementation stability typically does not matter

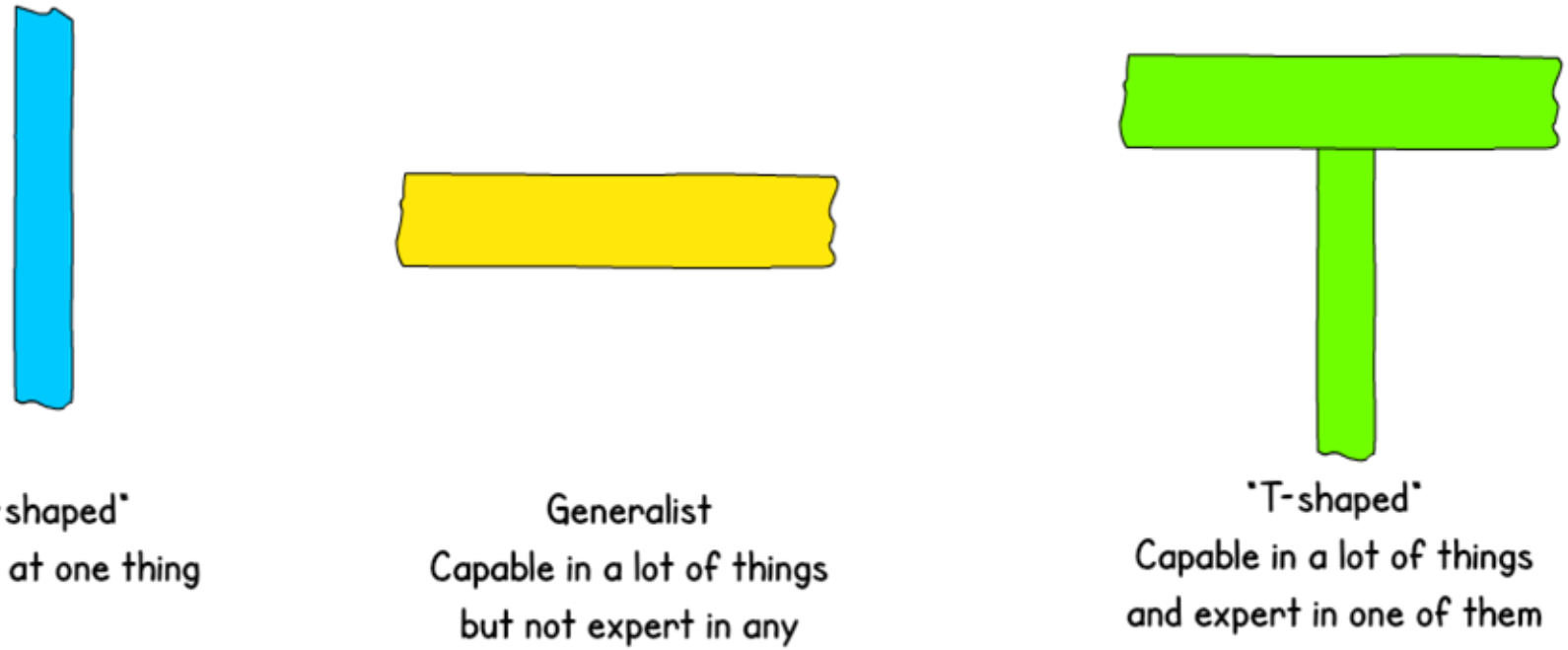
Software engineer

- Builds a product
- Concerned about cost, performance, stability, release time
- Identify quality through customer satisfaction
- Must scale solution, handle large amounts of data
- Detect and handle mistakes, preferably automatically
- Maintain, evolve, and extend the product over long periods
- Consider requirements for security, safety, fairness


Computer Science
ML
Math & Statistics
Unicorn
Software Engineering
Research
Domain Expertise

T-Shaped People

Broad-range generalist + Deep expertise



"I-shaped"
Expert at one thing

Generalist
Capable in a lot of things
but not expert in any

"T-shaped"
Capable in a lot of things
and expert in one of them

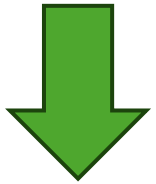
Latest Buzzword: π-Shaped People



Why familiar ML is necessary but insufficient

Mindset change…

Can the model perform the task?



Can this system use the model to achieve a goal under real constraints?

Model-centered question	AI Engineering question
Which model has the best aggregate score?	Which candidate supports the system goal and avoids unacceptable failures?
Does it work on the held-out set?	Does the evaluation represent current users, contexts, and consequential cases?
Can it generate a useful answer?	Should the system act on that answer, ask for confirmation, abstain, or escalate?
What is model inference latency?	What is the user-observed latency under load, including retrieval, tools, review, and retries?
How accurate is the model?	Which errors occur, who experiences them, and what happens next?
Does the prompt work in a demo?	How does it behave with untrusted input, conflicting instructions, tool access, and provider changes?
Can we improve the model?	Is model improvement the best use of effort, or would data, interface, architecture, or operations improve the product more?
Can we deploy it?	Can we monitor, investigate, update, and roll it back responsibly?

Quality Map

Is UniGuide better?

Improving answer quality by 2% but doubling latency and cost



Quality Map

- Model and AI behavior
  - task performance;
  - robust. & capability behavior;
  - explanation-related properties;
  - latency and resource.
- Data
  - correctness & completeness;
  - representativeness & freshness;
  - labeling quality & provenance;
- Pipeline
  - repeatability & testability;
  - traceability & versioning;
  - reproducibility & and rollback.
- System and operations
  - availability and e2e latency;
  - scalability & observability;
  - failure recovery; security and privacy; operating cost
- Intelligence experience
  - appropriate expectations;
  - understandability; user control; correction and undo;
  - human support & appeal
- Product and responsibility
  - user and organizational value;
  - accessibility; safety & avoidance of harm;
  - fairness; accountability;
  - legal and institutional acceptability;

Discussion

Project menu

1. Coding Assistant
2. Medical-conversation transcription and summary service
3. Customer-support agent with refund tools
4. Content-moderation and appeal system
5. A team-proposed AI product approved by the instructor

Round 1: define the product claim

- What changes in the world when the product succeeds?
- What changes when it fails?
- For **[intended user or stakeholder]**, the product uses **[AI capability]** to support **[decision or action]**, so that **[desired outcome]**.

Round 2: draw the system boundary

Draw arrows for information, control, and feedback

Round 3: inject failures

Round 4: choose three controls

Add one control before inference, one around the decision/action, and one after deployment

Possible control families

- data validation or source authority;
- model test or slice evaluation;
- structured output validation;
- confidence threshold or abstention;
- rule-based guardrail;
- permission boundary;
- preview and confirmation;
- human escalation;
- undo or appeal;
- telemetry and alert;
- canary release;
- version record and rollback.

Course content

1. From Models to AI-Enabled Systems
2. Requirements, Value, and Risk
3. Architecting ML Production Systems
4. Data Quality Engineering
5. Model Quality and Behavioral Testing
6. Testing and Experimenting at System Level
7. Intelligence Experience
8. Reproducible ML Pipelines and Deployment
9. Operations, Monitoring, and Responsible Evolution

How this course proceed…

- Students review the previous lecture
  - Review slides and do homework
  - Employ AI/Google
- Students prepare the next lecture
  - Reading assignment
  - Employ AI/Google
  - Prepare questions
- The class will be a discussion
  - Lecturer may introduce the content and set context
  - We will discuss!

Grading policy

- Lab/assignment/midterm: 40%
  - Homework, weekly
- Group project/final exam: 60%
  - Group of 4 students
  - Projects are provided by Instructor/TA or proposed by students
  - Work within 10 weeks: Weeks 2-10
  - Project report and presentation: Weeks 13-14

Summary

- Machine learning components are part of larger systems
- Data scientists and software engineers have different goals and focuses
  - Building systems requires both
  - Various qualities are relevant, beyond just accuracy
- Machine learning brings new challenges and intensifies old ones
- **Next class reading assignment**:
  - Ch. 4: When to Use Machine Learning
  - Ch. 5: Setting and Measuring Goals
  - Ch. 6: Gathering Requirements
  - Ch. 7: Planning for Mistakes